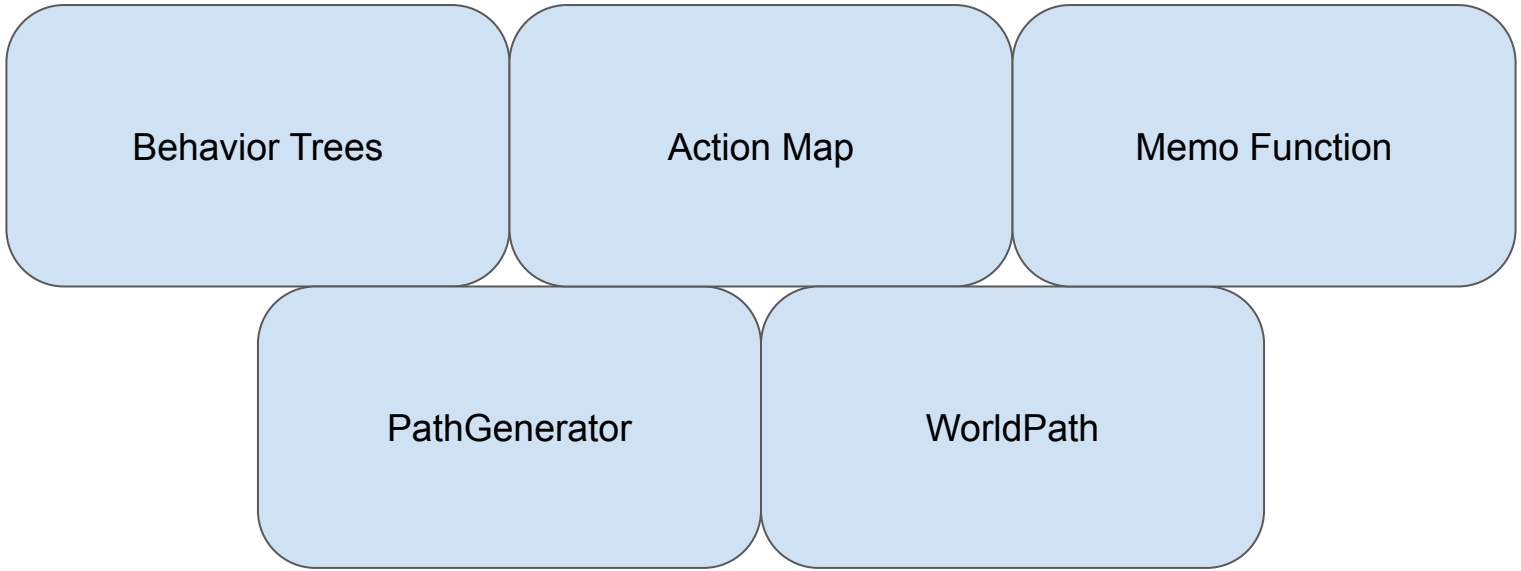
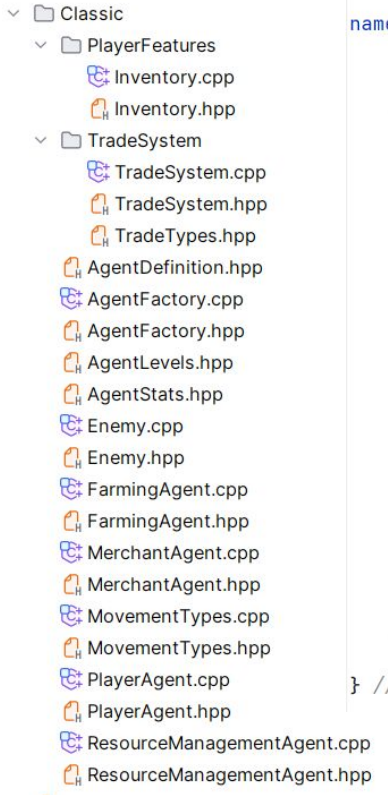
Group 2
Final Project Video

Logan, Jacob, Ty, Luke, and Henry

Recap of the 5 classes:



Our Agent Module




```cpp
namespace cse498 {
    struct WorldActions // just so they aren't globals
    {
        // Note 0 should ALWAYS Be stay based on AgentBase GetActionI
        static constexpr std::size_t REMAIN_STILL = 0; // "stay"
        static constexpr std::size_t MOVE_UP = 1; // "w"
        static constexpr std::size_t MOVE_DOWN = 2; // "s"
        static constexpr std::size_t MOVE_LEFT = 3; // "a"
        static constexpr std::size_t MOVE_RIGHT = 4; // "d"
        static constexpr std::size_t INTERACT = 5; // "e"
        static constexpr std::size_t QUIT = 6; // "q"

        static constexpr std::string REMAIN_STILL_STRING = "stay";
        static constexpr std::string MOVE_UP_STRING = "up";
        static constexpr std::string MOVE_DOWN_STRING = "down";
        static constexpr std::string MOVE_LEFT_STRING = "left";
        static constexpr std::string MOVE_RIGHT_STRING = "right";
        static constexpr std::string INTERACT_STRING = "e";
        static constexpr std::string QUIT_STRING = "q";
    };
} // namespace cse498
```

Agents Choose actions from the world actions:

- Interact: player interacts with surrounding agents governed by atk range and interaction range
- Movement commands decided by the agent's decision tree.

Structure of Agent Creation

Opted for a Factory Class:

- Decision Tree Nodes defined in static functions
- Creating the Agents themselves is ~5 lines

Simple Node example:

```
std::unique_ptr<Node> AgentFactory::AttackPlayer(const Enemy& enemy, WorldBase& world) {
    return TreeBuilder::Act( name: "Attack Player", func: [&world, &enemy](ExecutionContext&) ->Node::Status {
        auto* player = world.GetPlayer();
        if (player == nullptr || !player->IsAlive())
            return Failure;
        auto dmg :double = DamageCalculator::Calculate( attacker: enemy.GetStats(), defender: player->GetStats());
        player->TakeDamage(dmg);
        return Success;
    });
}
```

Simple Tree example:

```
auto root :unique_ptr<ContinuallyRepeat> = TreeBuilder::Repeat( name: "Example Root");
auto selector :unique_ptr<Selector> = TreeBuilder::Sel( name: "Example Behavior");
auto attackSeq :unique_ptr<Sequence> = TreeBuilder::Seq( name: "Attack Player Seq");

attackSeq->AddChild(IsPlayerInRange(enemy, world));
attackSeq->AddChild(AttackPlayer(enemy, [&] world));

selector->AddChild(std::move(attackSeq));
selector->AddChild(ChasePlayer(enemy, world));

root->SetChild(std::move(selector));

return root;
```

Agent Structure Cont.

Stats

Levels

```
    /**
     * Gets the stat object scaled by some leveling of the skeleton
     * @param level - level of the skeleton
     * @return stat object
     */
    static constexpr AgentStats GetSkeletonStats(size_t level) {
        double hp = 100 + 20 * static_cast<double>(level);
        double atk = 5 + 2 * static_cast<double>(level);
        double def = 5 + static_cast<double>(level);
        double range = 3 + static_cast<int>(level / 5);
        return {hp, atk, def, range, level};
    }
```

```
struct AgentStats {
    /// Starting Hp
    double mMaxHp = 0;
    /// Current HP (if you want it to be lower, then make enemy take damage on spawn
    double mHp = 0; // can have negative hp
    double mAtk = 0; // can have negative atk (would heal others)
    double mDef = 0; // can have negative def. Debuffs = extra dmg
    double mRange = 0; // range is positive
    size_t mLevel = 0; // level >= 0
    /**
```

Skeleton Behavior

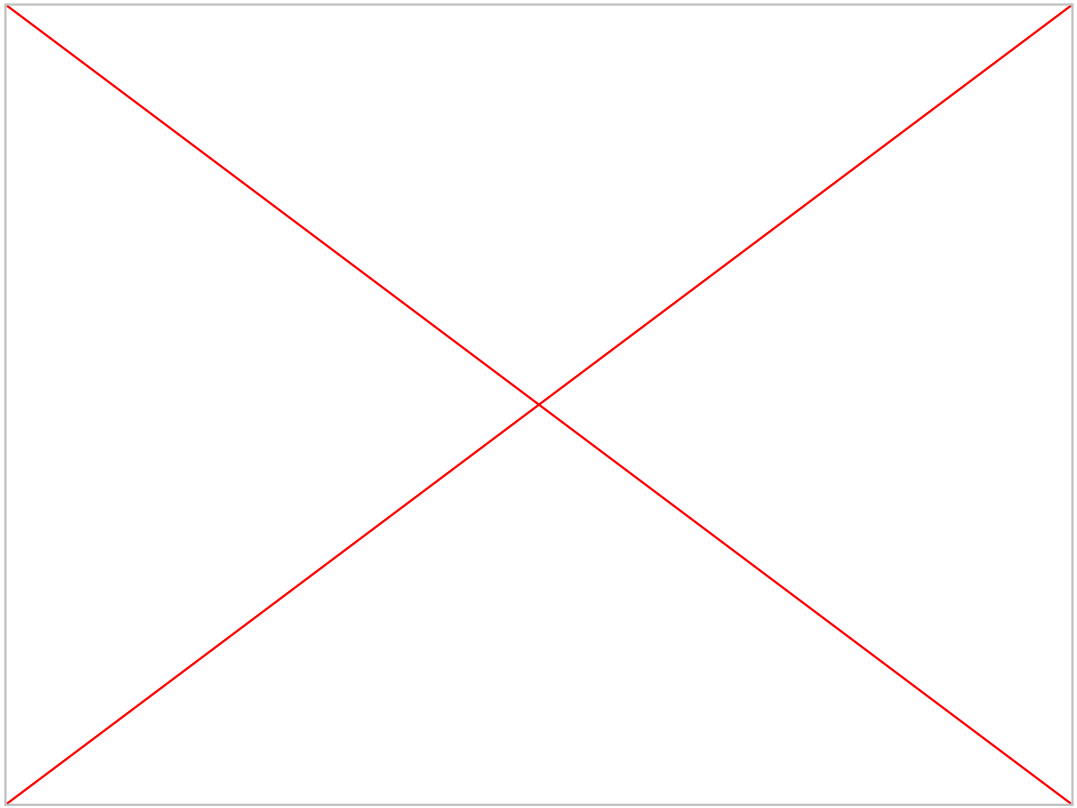
Skeleton has two main behaviors:

- Attacks player when safely within a bounded range (approx. 2.5–4 units) (depending on range and more complex calculation).
- Otherwise, moves closer or runs away strategically.
- It runs away for a maximum of 5 consecutive steps before pausing to attack again.

```
auto root :unique_ptr<ContinuallyRepeat> = TreeBuilder::Repeat( name: "Skeleton Root");
auto selector :unique_ptr<Selector> = TreeBuilder::Sel( name: "Skeleton Behavior");

auto atkSeq :unique_ptr<Sequence> = TreeBuilder::Seq( name: "Skeleton Attack Sequence");
atkSeq->AddChild(IsPlayerInBoundedRange(enemy, world));
atkSeq->AddChild(AttackPlayer(enemy, [&] world));
auto moveSeq :unique_ptr<Sequence> = TreeBuilder::Seq( name: "Skeleton Move");
moveSeq->AddChild(RangeChasePlayer(enemy, world));
moveSeq->AddChild(IsPlayerInRange(enemy, world)); // UNBOUNDED -- We are close enough an
moveSeq->AddChild(AttackPlayer(enemy, [&] world));

selector->AddChild(std::move(atkSeq));
selector->AddChild(std::move(moveSeq));
root->SetChild(std::move(selector));
return root;
```

Skeleton Behavior Example

We ported our world into the WebUI branch just to give a small demo with some color



Goblin Behavior

- Goblin uses a repeat → selector tree: each tick it tries to attack, then falls back to chase.
- Attacks when the player is within attack range and there is a clear line of sight on the grid.
- Otherwise chases: one step toward the player using shortest-path planning through the world grid; movement maps to the registered world actions.

```
auto root :unique_ptr<ContinuallyRepeat> = TreeBuilder::Repeat( name: "Goblin Root");
auto selector :unique_ptr<Selector> = TreeBuilder::Sel( name: "Goblin Behavior");
auto attackSeq :unique_ptr<Sequence> = TreeBuilder::Seq( name: "Goblin Attack Seq");
attackSeq->AddChild(IsPlayerInRange(enemy, world));
attackSeq->AddChild(AttackPlayer(enemy, [&] world));

selector->AddChild(std::move(attackSeq));
selector->AddChild(ChasePlayer(enemy, world));

root->SetChild(std::move(selector));

return root;
```

Weapons

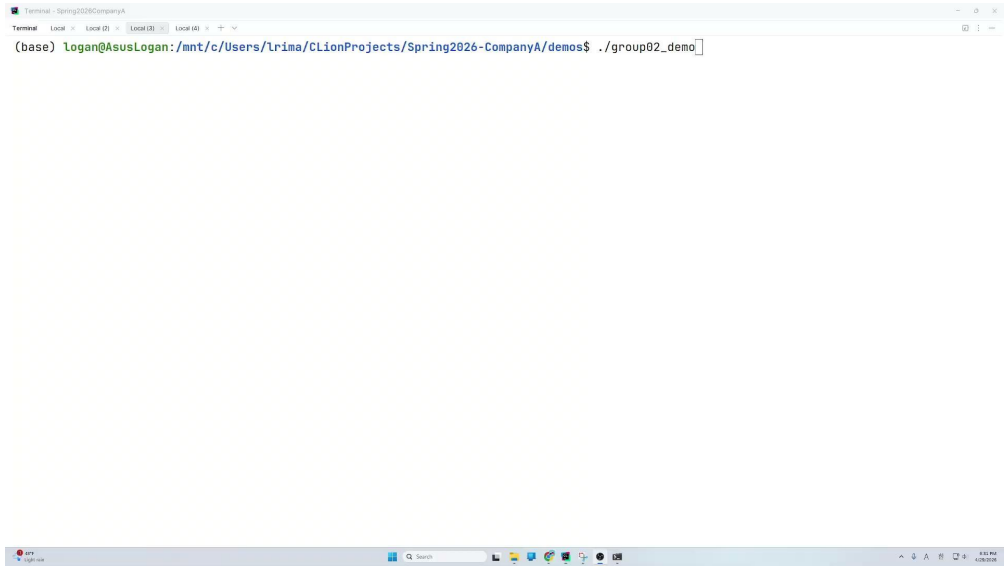
When the player's stats are set:

- They are saved as the base stats and upon hand inventory hotbar index movement then the RefreshCombatFromHand is called (not shown)

```
namespace cse498 {
class ItemWeapon : public Item {
private:
    double m_range; // Range of the weapon (in tiles)
    double m_damage; // Amount of damage the weapon can do
    double m_hit_bonus; // Bonus the weapon has to hit
```

```
void PlayerAgent::SetStats(const AgentStats& stats) {
    mBaseCombatStats = stats;
    RefreshCombatFromHand();
}

void PlayerAgent::RefreshCombatFromHand() {
    mStats = mBaseCombatStats;
    const Item* hand = mInventory.GetHand();
    if (hand == nullptr || !hand->IsWeapon()) {
        return;
    }
    auto* weapon = dynamic_cast<const ItemWeapon*>(hand);
    if (weapon == nullptr || weapon->IsTool()) {
        return;
    }
    mStats.mAtk = mBaseCombatStats.mAtk + weapon->GetDamage();
    mStats.mRange = weapon->GetRange();
}
```

Weapons Demo



Merchant Agent:

- An NPC the player can trade with
- Tracks:
  - Shop availability
  - Trade offers
  - Gold
  - Greeting / Closed messages
- Supports both limited-stock and unlimited-stock offers

```
class MerchantAgent : public AgentBase {
private:
    /** Controls whether merchant currently accepts trades */
    bool mAvailableForTrade = true;

    /** Catalog of items merchant can trade*/
    std::vector<TradeOffer> mOffers;

    /** Currency held by merchant for paying the player for sales. */
    std::size_t mGold = 0;


protected:
    std::string mTradeGreeting = "Take a look at my shop.";
    std::string mTradeClosedMessage = "Sorry, shop's closed.";
```

```
/**
 * Attempts to buy an item from the merchant into the player's inventory.
 *
 * @param player Player performing the purchase
 * @param itemName Name of the item to buy
 * @param quantity Quantity requested
 * @return Result describing success or failure and gold movement
 */
TradeResult BuyFromMerchant(PlayerAgent& player, const std::string& itemName, std::size_t quantity = 1);

/**
 * Attempts to sell an item from the player to the merchant. If the item is not already
 * sold by the merchant, a new limited offer is created so the item can appear in the shop afterward.
 *
 * @param player Player performing the sale
 * @param itemName Name of the item to sell
 * @param quantity Quantity requested
 * @return Result describing success or failure and gold movement
 */
TradeResult SellToMerchant(PlayerAgent& player, const std::string& itemName, std::size_t quantity = 1);
```

TradeSystem

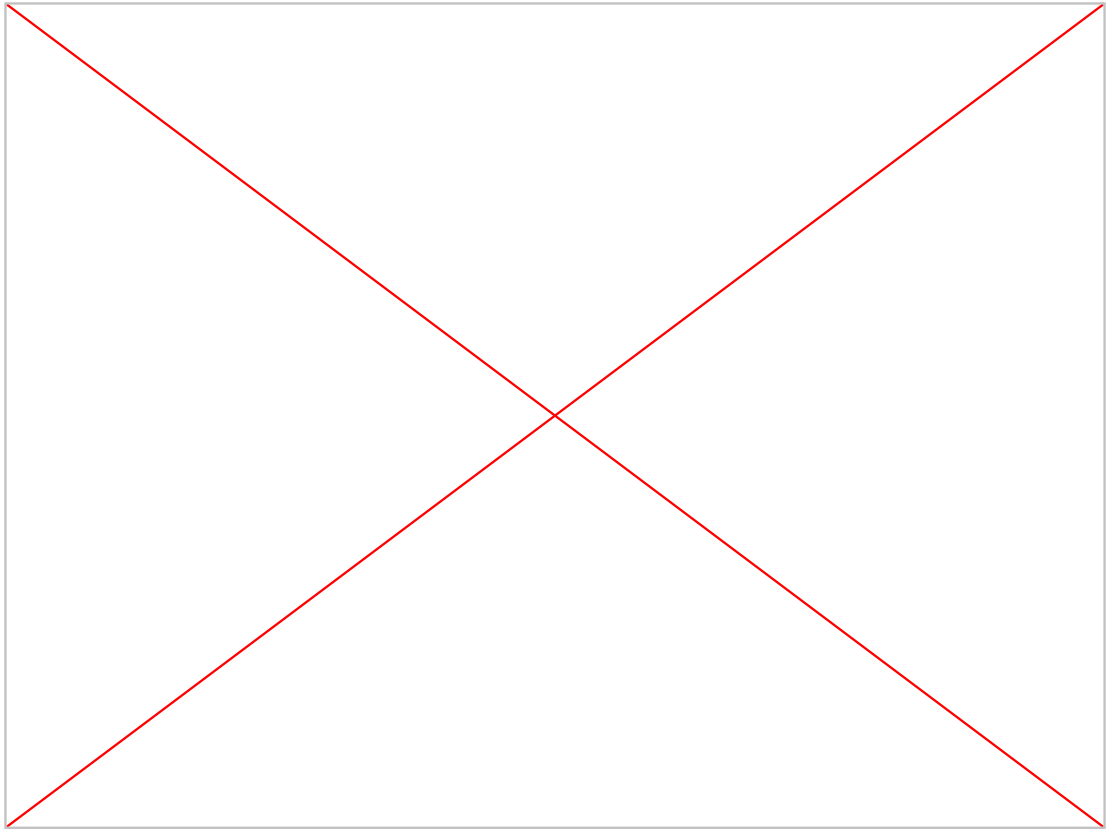
- Keeps transaction rules separate from merchant
- Checks:
  - Valid Quantity
  - Item exists
  - Player gold/items
  - Merchant stock/gold
  - Inventory has room
- Failed trades return a TradeResult instead of partially changing state

```
if (quantity == 0) {
    return {TradeStatus::InvalidQuantity, "Quantity must be at least 1.", offer.mItemName, 0, 0};
}

if (!offer.HasEnough(quantity)) {
    return {TradeStatus::MerchantOutOfStock, "Merchant is out of stock.", offer.mItemName, quantity, 0};
}

const std::size_t totalCost = offer.mBuyPrice * quantity;
if (player.GetGold() < totalCost) {
    return {TradeStatus::InsufficientFunds, "You do not have enough gold.", offer.mItemName, quantity,
            totalCost};
}

if (!player.SpendGold(totalCost)) {
    return {TradeStatus::InsufficientFunds, "Could not remove player gold.", offer.mItemName, quantity,
            totalCost};
}
```

```
struct TradeResult {
    TradeStatus mStatus = TradeStatus::Success;
    std::string mMessage;
    std::string mItemName;
    std::size_t mQuantity = 0;
    std::size_t mGoldMoved = 0;

    [[nodiscard]] bool IsSuccess() const { return mStatus == TradeStatus::Success; }
};
```

Merchant Example

Sadly, all the UI required for MerchantAgent was not fully complete by the UI groups so trading behavior is demonstrated through the Group 2 demo flow instead.



Farming Agent

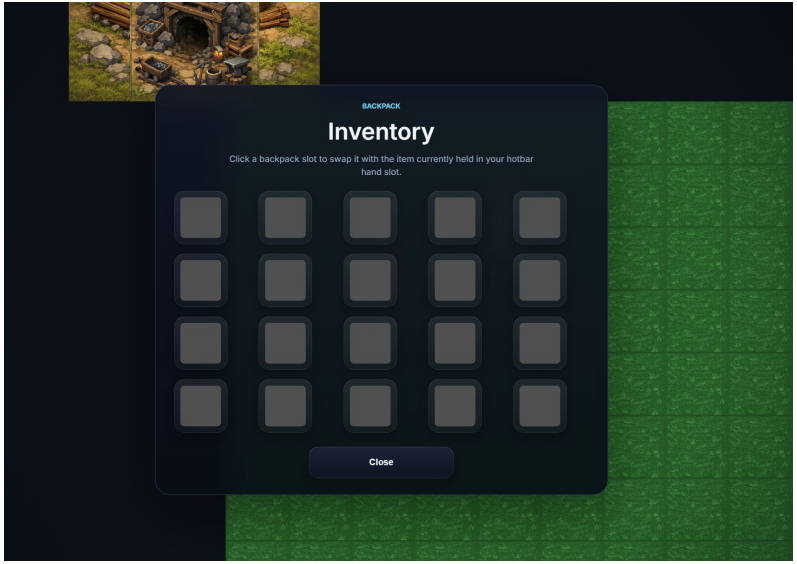
- Extends MerchantAgent
- Adds worker NPC behvior:
  - Home position
  - Assigned building
  - Work interval
  - Restock item / amount
- Designed for Interactive World integration, but not shown in demo due to the decision to use FetchAgent.

```
class FarmingAgent : public MerchantAgent {
public:
    /// States the farmer can be in
    enum class FarmerState { IdleAtHome, GoingToWork, Working, ReturningHome };

private:
    /// Farmer assigned building
    Building* mAssignedBuilding = nullptr;

    /// Where farmer usually is
    WorldPosition mHomePosition{0, 0};

    /// Current farmer state
    FarmerState mState = FarmerState::IdleAtHome;

    /// Time since last worked
    int mTicksSinceWork = 0;
    /// How much time should pass between work
    int mWorkInterval = 8;

    /// Amount to restock limited offers
    int mRestockAmount = 2;
    /// Item to restock after work
```

Inventory System

The inventory system:

- 10 slot hotbar 4x5 backpack
- unordered_map insertion/deletion memory
- Built on std::array




```
Inventory:
HOTBAR: empty (I:sword, Q:1) empty empty empty empty empty empty empty empty
Row 1:empty empty empty empty empty
Row 2:empty empty empty empty empty
Row 3:empty empty empty empty empty
Row 4:empty empty empty empty empty
```

```
 * @param item - item to add --
 * @return size_t = amount remaining to be added. 0 == everyt
 */
size_t AddItem(std::unique_ptr<Item>, size_t quantity = 1);
template<typename ITEM_T, typename... Args>
    requires std::derived_from<ITEM_T, Item>
// it may be better to replace forwarding to 2 args
size_t AddItem(size_t quantity = 1, Args&&... args)
size_t RemoveItem(const std::string& name, size_t amount = 1, bool isAllOrNothing = false);
    std::array<InventorySlot, INVENTORY_SIZE> mInventory{};

    /// This is for constant speed lookup -- a little overkill but nice
    /// key: string name --> Value: index locations in array
    /// Ordered for consistency in removal
    std::unordered_map<std::string, std::set<size_t, CircularCompare>> mItemMap;
```

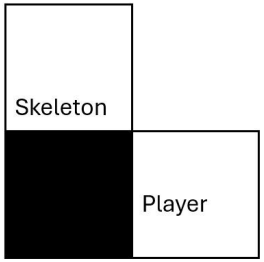
PathGenerator Tools -- IsPathClear

New Tool to determine if a path from the start along the path vector hits no walls.

- Built on parameterization functions to determine next tile boundaries

```
// 1. startX + V_X * t = 3
// 2. startX + V_X * t = 4
// solve for the difference in 't' = delta T. (eq2 - eq1)
double tDeltaX = (path.X() != 0) ? std::abs( x: 1 / path.X()) : INFINITY;
double tDeltaY = (path.Y() != 0) ? std::abs( x: 1 / path.Y()) : INFINITY;

// This part is really simple we are just solving for 't' in:
//  startX + V.X() * t = floor(startX) + 1
// from start position how much of time step to reach the next barrier (depending on direction)
double nextBoundaryX = (stepX > 0) ? (tileX + 1) : tileX;
double nextBoundaryY = (stepY > 0) ? (tileY + 1) : tileY;
double tMaxX = (path.X() != 0) ? (nextBoundaryX - startShifted.X()) / path.X() : INFINITY;
double tMaxY = (path.Y() != 0) ? (nextBoundaryY - startShifted.Y()) / path.Y() : INFINITY;
```

Hit FAILS




```
/**
 * from the start position follows the path vector returning all points
 * @param start - start position
 * @param path - path as vector to follow
 * @param request - other information to verify path
 * @return true if path is clear
 */
static bool IsPathClear(const WorldPosition& start, const PathVector& path, const PathRequest& request);
```

PathGenerator -- FindFurthestPoint

New Function for Skeleton to run away

- Finds vector in 8 cardinal directions away from ‘center’ →
- Travels in 2-3 of those directions only with DFS limited to 5 depth iterations.
    - Efficient and fast for determining a good escape plan

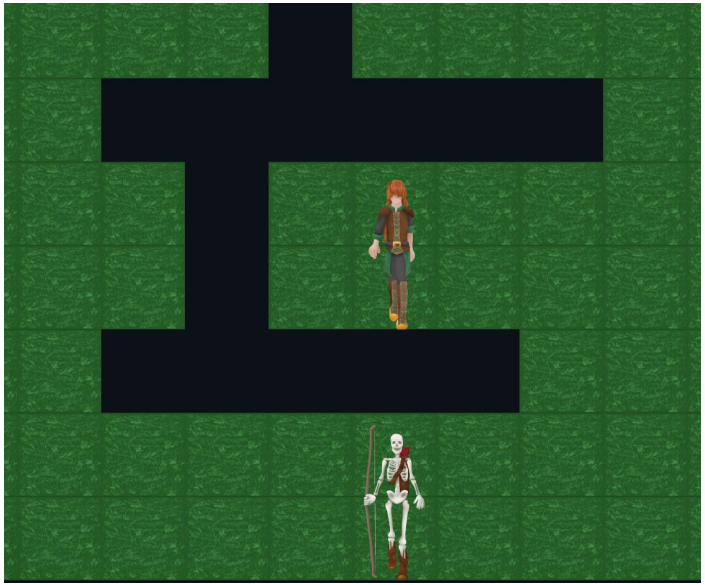
```
if (static_cast<bool>(dx) && static_cast<bool>(dy))
{
    dirs[0] = {x: dx, y: 0};
    dirs[1] = {x: 0, y: dy};
    dirCount = 2;
} else {
    dirCount = 3;
    if (static_cast<bool>(dx)) {
        dirs[0] = {x: dx, y: 0};
        dirs[1] = {x: 0, y: STEP}; // this is hard
        dirs[2] = {x: 0, y: -STEP};
    } else {
        dirs[0] = {x: 0, y: dy};
        dirs[1] = {x: -STEP, y: 0};
        dirs[2] = {x: STEP, y: 0};
    }
}
```

```
/**
 * A simpler custom A* operating under different conditions and assumptions to be more efficient
 * on finding a goal point in the provided general direction
 * RETURNS 4 DIRECTIONS ONLY for operation
 * @param start Start position
 * @param direction - allowed travel direction (1,1), (-1,1), (1,-1), (-1,-1)
 * @param maxRange - max search range for efficiency
 * @return vector of world positions to get to the furthest point
 * If You can't move!! THEN EMPTY!
 */
static std::vector<WorldPosition> FindFurtherestPoint(const WorldPosition& start, const WorldPosition& center,
                                                      const PathVector& direction, const PathRequest& request,
                                                      double maxRange);
```

Final Recap:

Our Agent Module does what classic agents are supposed to with decent structure and expandability.

- We have Skeleton, goblin, merchants with expected behaviors and some unique quirks
- Everyone ran short on time, so not everything came together perfectly, but it was fun.



The behavior looks somewhat weird but was completely intentional, it just didn't play out exactly like this in my brain. When skeleton is in range and out of sight he waits and gives the player a break